Fullstack Development

Fullstack Landscape

> Diagram

Inspired from this VDO.

Part 1: The battle of the architectures

> MPA vs SPA vs HTMX

Multi-page application (MPA)

> The web's default architecture (1991–2000s)

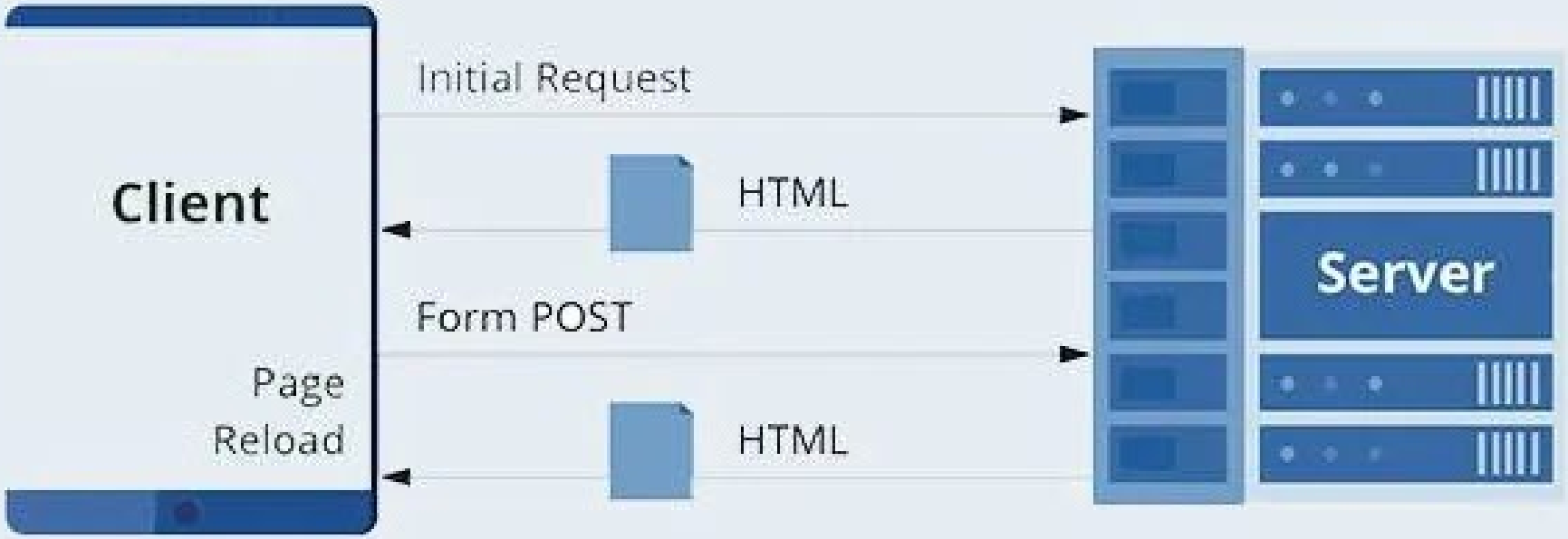
History

- `1990–91`: Tim Berners-Lee's WWW — static hypertext documents only
- `1993`: CGI — first standard way for a server to generate HTML dynamically
- `1995–99`: PHP, ASP, JSP — server-side scripting goes mainstream
- `2004–05`: Rails, Django — opinionated MVC frameworks standardize the pattern

Multi-page application

- Loads a new page every time you perform an action.
- Traditional web applications.
- Use server-side technologies
  - PHP, Ruby on Rails, ASP.NET, Java, and Node JS.
- Can include JavaScript (`script`) for client-side interactivity

MPA Lifecycle



Todo app (MPA)

- `Express JS` + `Pug` (renderer)
- `git clone https://github.com/fullstack-69/ls-mpa.git`

Endpoint

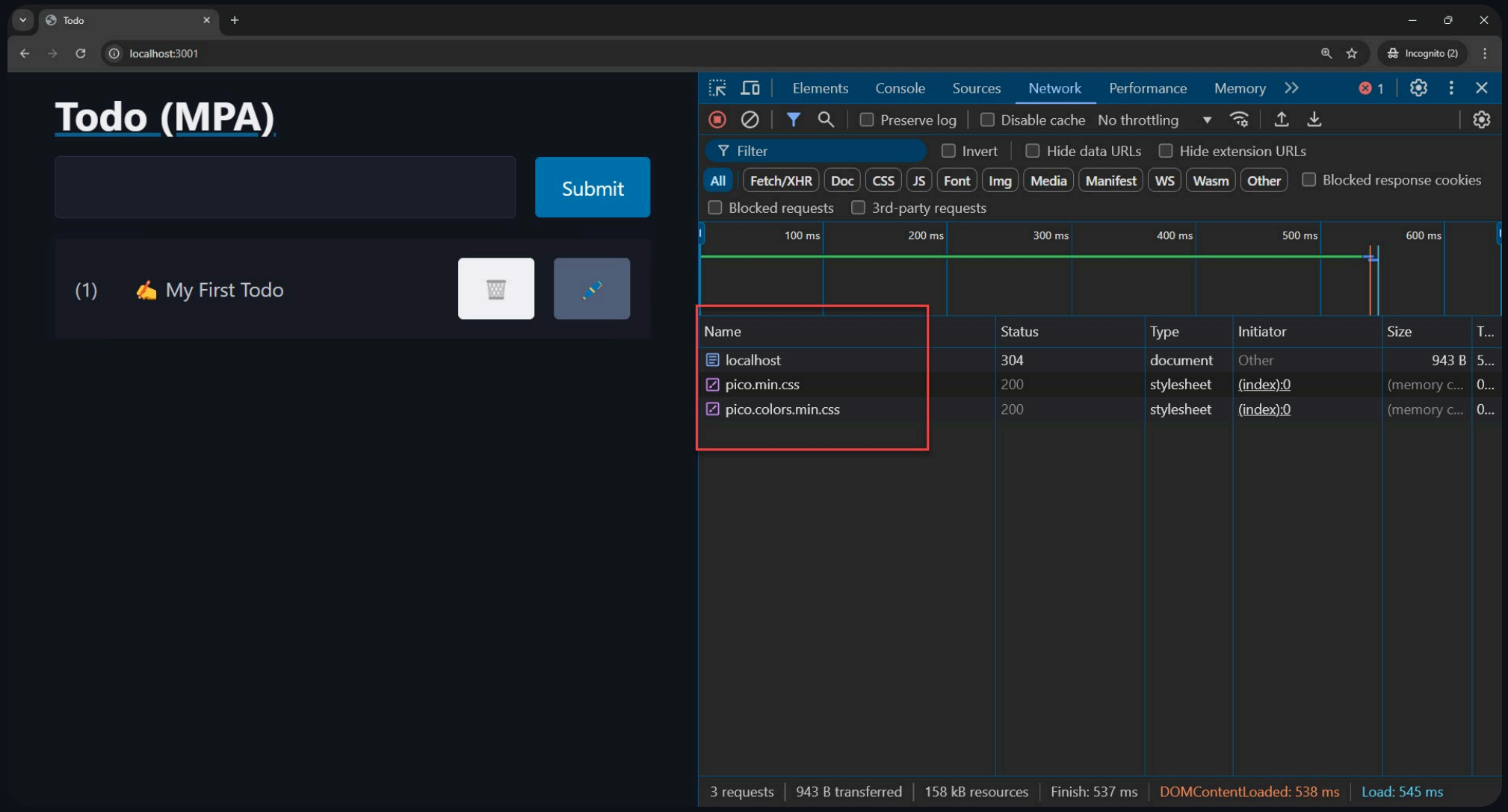
./src/index.ts

```
app.get("/", async (req, res) => {
  const message = req.query?.message ?? "";
  const todos = await getTodos();
  // Output HTML
  res.render("pages/index", {
    todos,
    message,
    mode: "ADD",
    curTodo: { id: "", todoText: "" },
  });
});
```

Renderer

`./view/pages/index.pug`

```
body
    main(class="container")
        a(href="/")
            h1 Todo (MPA)
        div(id="todoform")
            include ../components/inputform.pug
        div(id="todolist")
            include ../components/todolist.pug
```

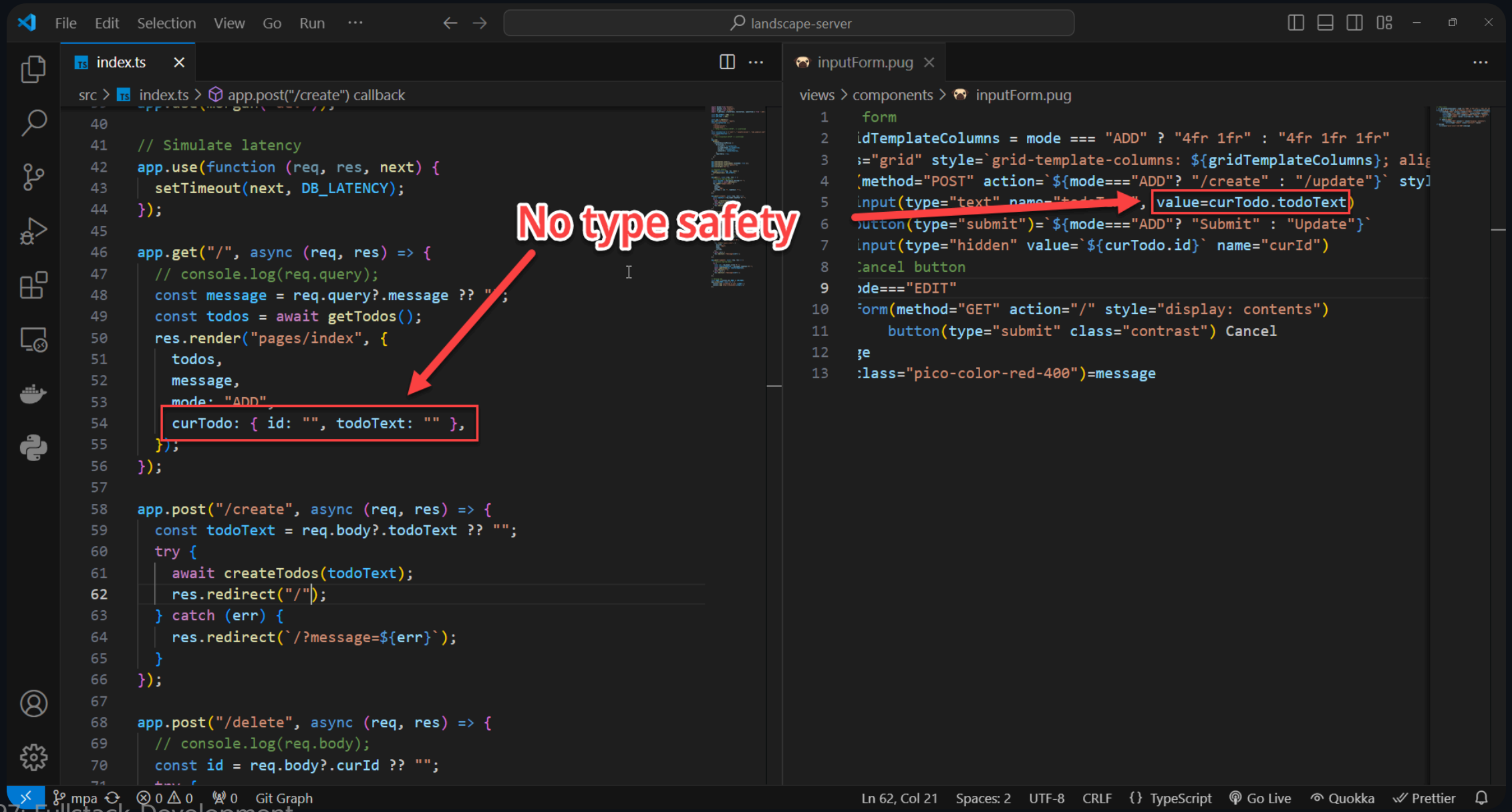


Use incognito mode to avoid loading chrome extensions.

Note

- Every button is wrapped in a separate form
  - Need to trigger different endpoints.
- Need to use `input(type="hidden")` to encode additional information.

```
form(action="/delete" method="POST" style="display: contents")
  input(type="hidden" value=`${todo.id}` name="curId")
  button(type="submit" class="contrast" style="margin-bottom: 0") 🪣
form(action="/edit" method="POST" style="display: contents")
  input(type="hidden" value=`${todo.id}` name="curId")
  button(type="submit" class="secondary" style="margin-bottom: 0") 🖌️
```


File Edit Selection View Go Run
landscape-server
index.ts
inputForm.pug
src > index.ts > app.post("/create") callback
views > components > inputForm.pug
No type safety
// Simulate latency
app.use(function (req, res, next) {
setTimeout(next, DB_LATENCY);
});
app.get("/", async (req, res) => {
// console.log(req.query);
const message = req.query?.message ?? "";
const todos = await getTodos();
res.render("pages/index", {
todos,
message,
mode: "ADD",
curTodo: { id: "", todoText: "" },
});
});
app.post("/create", async (req, res) => {
const todoText = req.body?.todoText ?? "";
try {
await createTodos(todoText);
res.redirect("/");
} catch (err) {
res.redirect(`/?message=${err}`);
}
});
app.post("/delete", async (req, res) => {
// console.log(req.body);
const id = req.body?.curId ?? "";
value=curTodo.todoText
mpa
Git Graph
Ln 62, Col 21
Spaces: 2
UTF-8
CRLF
TypeScript
Go Live
Quokka
Prettier

MPA - the good

- Simple code
- Ship less JavaScript
- Good for SEO
- State in URL

MPA - the bad

- Page reload
- No spinner
- No element disabling
- No hot reloading
- No type safety

Single-page application (SPA)

> The "thick client" era (2004–2010s)

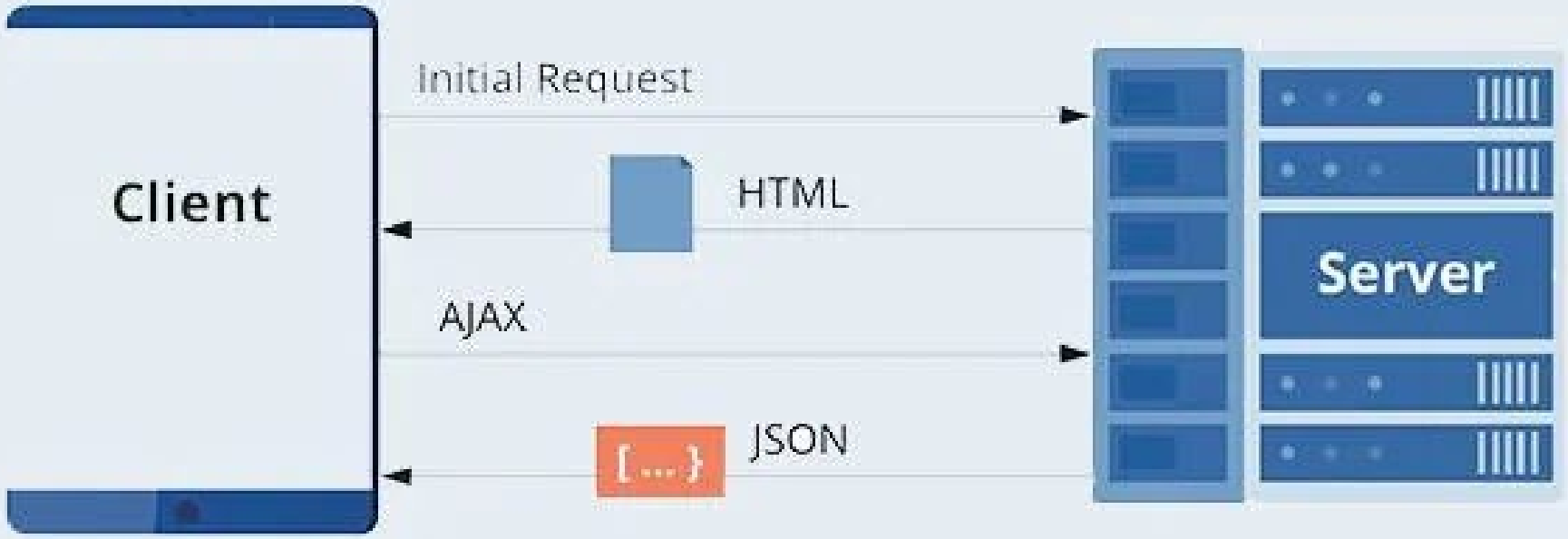
History

- `2004`: Gmail launches — async UI updates without page reloads
- `Feb 2005`: Google Maps launches; Jesse James Garrett coins "Ajax"
- `2006`: jQuery — makes AJAX/DOM manipulation practical for everyone
- `2010–14`: Backbone/Knockout → AngularJS (2010) → React (2013) → Vue (2014)

Single-page application

- Single-page application
  - Loads a single HTML page and dynamically updates the content as the user interacts with the app.
- Use frontend and backend frameworks separately.

SPA Lifecycle

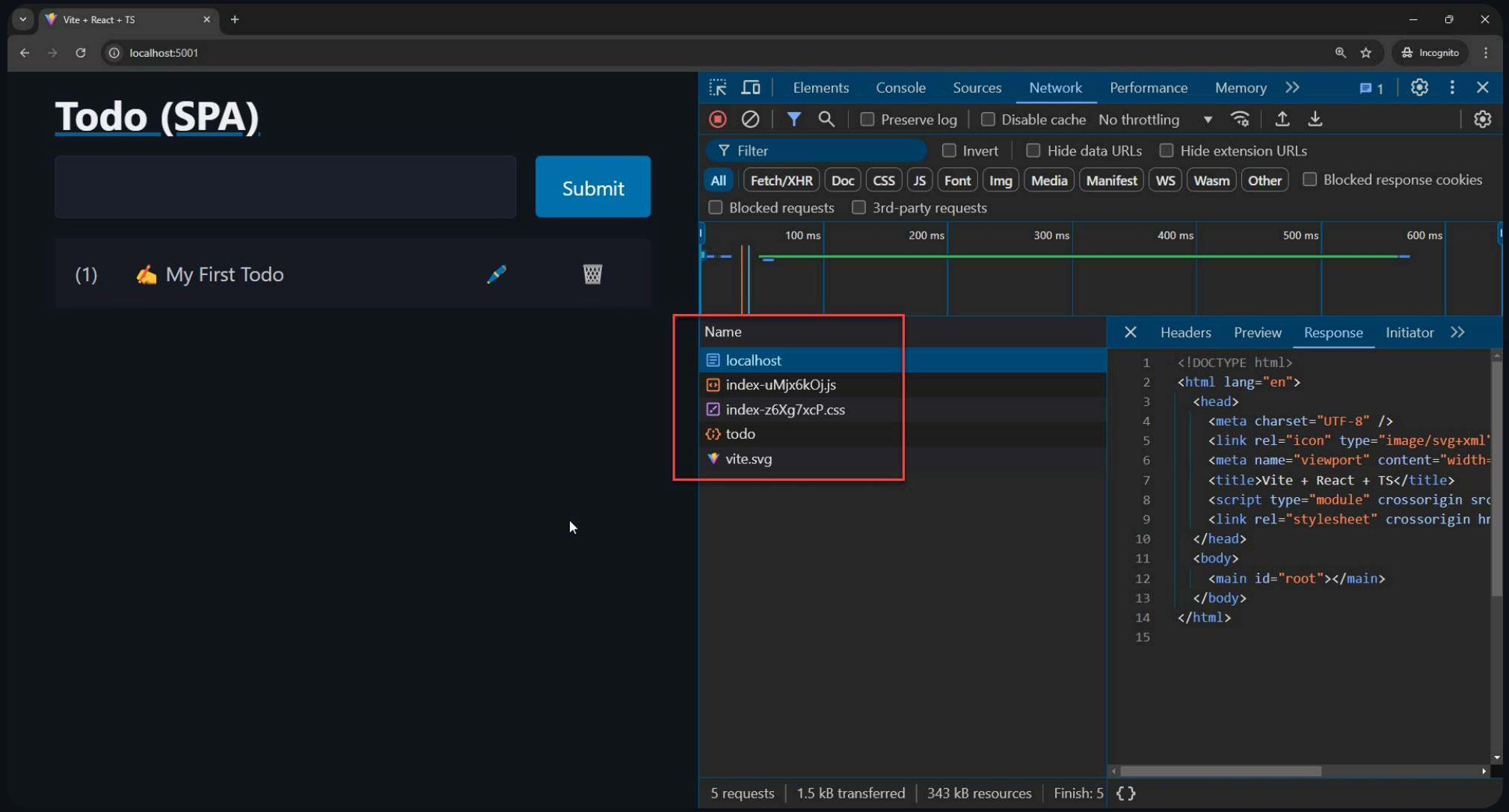


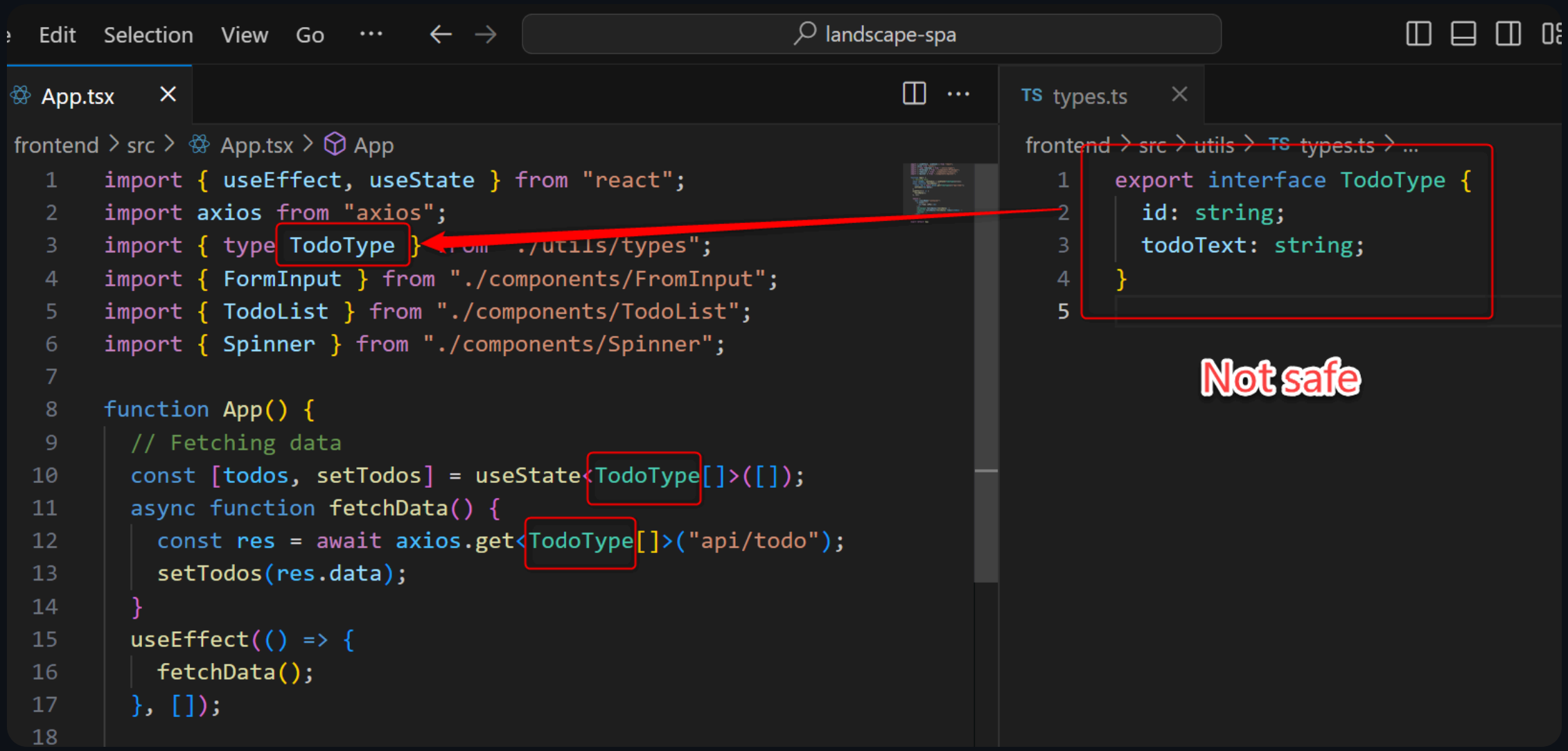
Todo app (SPA)

- `Express JS` + `React`

SPA - Todo

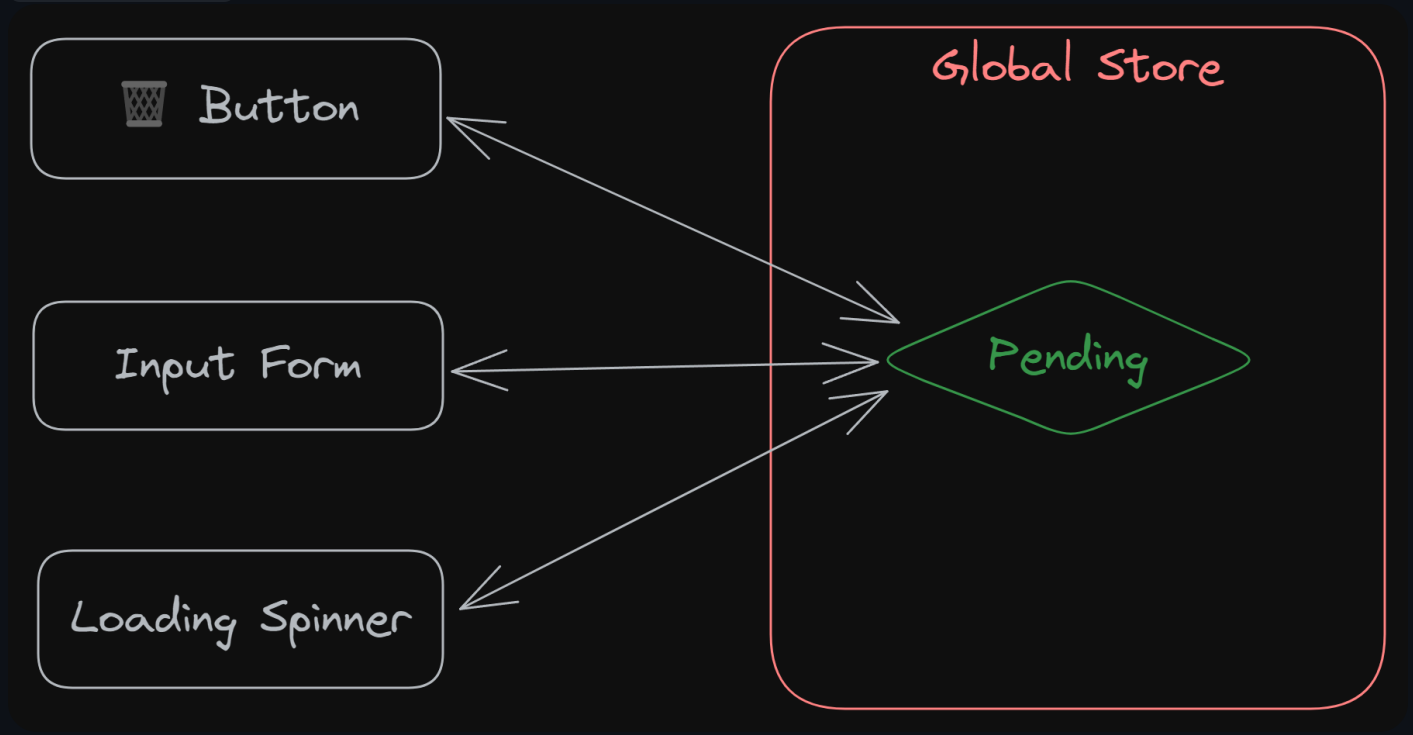
- `Express JS` + `React`
- `git clone https://github.com/fullstack-69/ls-spa.git`


Vite + React + TS
localhost:5001
Incognito
Todo (SPA)
Submit
(1) My First Todo
Elements
Console
Sources
Network
Performance
Memory
Preserve log
Disable cache
No throttling
Filter
Invert
Hide data URLs
Hide extension URLs
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
WS
Wasm
Other
Blocked response cookies
Blocked requests
3rd-party requests
100 ms
200 ms
300 ms
400 ms
500 ms
600 ms
Name
localhost
index-uMjx6kOj.js
index-z6Xg7xcP.css
todo
vite.svg
Headers
Preview
Response
Initiator
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<link rel="icon" type="image/svg+xml"
<meta name="viewport" content="width=
<title>Vite + React + TS</title>
<script type="module" crossorigin src
<link rel="stylesheet" crossorigin hr
</head>
<body>
<main id="root"></main>
</body>
</html>
5 requests
1.5 kB transferred
343 kB resources
Finish: 5


Edit Selection View Go
landscape-spa
App.tsx
frontend > src > App.tsx > App
import { useEffect, useState } from "react";
import axios from "axios";
import { type TodoType } from "./utils/types";
import { FormInput } from "./components/FromInput";
import { TodoList } from "./components/TodoList";
import { Spinner } from "./components/Spinner";
function App() {
// Fetching data
const [todos, setTodos] = useState<TodoType[]>([]);
async function fetchData() {
const res = await axios.get<TodoType[]>("api/todo");
setTodos(res.data);
}
useEffect(() => {
fetchData();
}, []);
types.ts
frontend > src > utils > types.ts > ...
export interface TodoType {
id: string;
todoText: string;
}
Not safe

Global store

- `zustand`



SPA - the good

- No page refresh
- Spinner
- Element disabling
- Hot reloading
- Type safety

SPA - the bad

- More code
- 2 frameworks
- More complexity
- More JavaScript loaded
- No HTML content (SEO)
- State not in URL

HTMX

> The hypermedia revival (2013–present)

What exactly is HTMX?

> Small JS library that can swap parts of UI with ***HTML response*** from a server.

#1 2024 JavaScript Rising Stars

History

- `2013` : Carson Gross builds Intercooler.js to fix slow client-side table sorting
  - Users tell him: "this is RESTful" → sends him back to Roy Fielding's dissertation
  - Fielding's 2008 essay: most "REST APIs" ignore HATEOAS (hypermedia constraint)
- `2020` : rewritten and relaunched as htmx — drops jQuery dependency
- `2022–23` : viral adoption, framed as reaction to SPA/build-tooling fatigue

HTMX - Todo

- `Express JS` + `Pug`
- `git clone https://github.com/fullstack-69/ls-htmx.git`

HTMX - the good

- Less code than SPA
- Less complexity than SPA
- SSR enabled (SEO-friendly)
- No reloading
- Spinner enabled
- Element disabling enabled
- State in URL enabled

HTMX - the bad

- No hot reloading
- No type safety

In addition

- Hypermedia as the Engine of Application State (HATEOAS)
- HTMX + Alpine.js
- You can be the CEO of HTMX.
- There would never be an HTMX 3.0.

App Comparison

UX

Item	MPA	SPA	HTMX
No page-refresh	❌	✅	✅
Spinner	❌	✅	✅
Element disabling	❌	✅	✅

App comparison (technical)

Item	MPA	SPA	HTMX
Amount of `JS` loaded	✅	❌	✅
HTML content (SEO)	✅	❌	✅
State in URL	✅	❌	✅

DX

Item	MPA	SPA	HTMX
Number of frameworks	1 ✅	2 ❌	1 ✅
Complexity	Less ✅	More ❌	Less ✅
Lines of code	Less ✅	More ❌	Less ✅
Type Safety	Less ❌	More ✅	Less ❌
Hot reloading	Partial ❌	Full ✅	Partial ❌

Code Count

Architecture	Lines
MPA	~180
HTMX	~260
SPA	~420

Summary

Architecture	Usage
MPA	Backend-critical (financial, ERP, ทะเบียน)
SPA	Dashboards, editors
HTMX	Internal Apps

Appendix

> Code walkthroughs: MPA → SPA → HTMX.
> This part is AI-generated and is not used for tutorials.

Shared Foundation

All three apps simulate the same 1-second DB latency — making UX differences undeniable in a live demo:

```
// src/index.ts — identical in all three projects
const DB_LATENCY = 1000;
app.use(function (req, res, next) {
  setTimeout(() => next(), DB_LATENCY);
});
```

Every button press visibly stalls for a second. This is the forcing function that makes the UX comparison real, not hypothetical.

MPA: Project Structure

```
ls-mpa/
├── src/
│   ├── db.ts          # In-memory CRUD store
│   └── index.ts       # Express server — 5 endpoints
└── views/
    ├── pages/
    │   └── index.pug          # Full page layout
    └── components/
        ├── inputForm.pug      # Add/Edit form (mode-aware)
        └── todoList.pug       # List with hidden-input buttons
```

One server. One template engine. No browser JavaScript.

MPA: Route Pattern

Every action → POST to a dedicated endpoint → `res.redirect("/")`:

```
// src/index.ts
app.post("/create", async (req, res) => {
  await createTodos(req.body?.todoText ?? "");
  res.redirect("/"); // full page reload
});

app.post("/delete", async (req, res) => {
  await deleteTodo(req.body?.curId ?? "");
  res.redirect("/"); // full page reload
});
```

The browser re-fetches everything — HTML, CSS, the entire todo list — on every mutation.

MPA: The Hidden-Input Pattern

HTML forms can only send their own fields. To attach a todo's `id` to a delete/edit button, each button gets its own `<form>` with a hidden input:

```
//- views/components/todoList.pug
form(action="/delete" method="POST" style="display: contents")
  input(type="hidden" value=`${todo.id}` name="curId")
  button(type="submit") 🪢

form(action="/edit" method="POST" style="display: contents")
  input(type="hidden" value=`${todo.id}` name="curId")
  button(type="submit") 🖌️
```

No JavaScript. Each button fires a separate form submit to a different endpoint.

MPA: The Edit Flow

Clicking ✏️ re-renders the same page in EDIT mode with the selected todo pre-filled:

```
// src/index.ts
app.post("/edit", async (req, res) => {
  const curTodo = todos.find((el) => el.id === id);
  res.render("pages/index", {
    mode: "EDIT", // ← toggles form UI
    curTodo, // ← pre-fills the input
    todos,
  });
});
```

The template reads `mode` to choose between "Submit"/"Update" labels and whether to show a Cancel button.

MPA: Full Page Layout

```
//- views/pages/index.pug
body
  main(class="container")
    a(href="/")
      h1 Todo (MPA)
    div(id="todoform")
      include ../components/inputform.pug
    div(id="todolist")
      include ../components/todolist.pug
    //- Livereload
    //- script(type='text/javascript' src="script.js")
```

The commented-out livereload script is a sign that someone tried to add dev-time browser refresh and gave up — a characteristic limitation of the plain server-rendering workflow.

SPA: Project Structure

```
ls-spa/
├── backend/src/
│   ├── db.ts          # Same in-memory store
│   └── index.ts       # JSON API: GET/PUT/PATCH/DELETE /todo
└── frontend/src/
    ├── main.tsx             # React entry point
    ├── App.tsx              # Root: fetches todos, composes UI
    ├── components/
    │   ├── FromInput.tsx    # Form with manual pending state
    │   ├── TodoList.tsx     # List with Axios delete/edit
    │   └── Spinner.tsx      # Reads pending from global store
    └── utils/
        ├── store.ts         # Zustand: mode, pending, inputText, curId
        └── types.ts         # Shared TodoType
```

Two separate codebases. Two `pnpm install` commands. Two servers in dev.

SPA: The Empty Shell

`frontend/index.html` — the entire initial HTML sent to the browser:

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.tsx"></script>
</body>
```

This is what a search crawler — and a user on a slow connection — sees before JavaScript executes. No content. No SEO.

SPA: JSON REST API

`backend/src/index.ts` — proper HTTP verbs, JSON responses:

```
app.get("/todo", async (req, res) => res.json(await getTodos()));
app.put("/todo", async (req, res) => {
  /* create */ res.json({ msg: "..." });
});
app.delete("/todo", async (req, res) => {
  /* delete */ res.json({ msg: "..." });
});
app.patch("/todo", async (req, res) => {
  /* update */ res.json({ msg: "..." });
});
```

The backend is fully decoupled from the UI. It knows nothing about how data will be displayed.

SPA: Global State (Zustand)

`frontend/src/utils/store.ts` — one store for all UI state:

```
interface TodoStoreState {
  mode: "ADD" | "EDIT";
  curTodo: TodoType;
  pending: boolean; // drives input/button disabling
  inputText: string;
  curId: string;
  // ... setters
}
```

`pending` is the key field: when `true`, inputs and buttons are disabled. This is the manual equivalent of HTMX's declarative `hx-disabled-elt`.

SPA: Manual Loading State

`frontend/src/components/FromInput.tsx`:

```
function handleSubmit() {
  setPending(true); // ← manually disable UI
  axios
    .request({ url: "/api/todo", method: "put", data: { todoText: inputText } })
    .then(fetchData) // ← re-fetch full todo list
    .then(() => {
      setInputText("");
      setMessage("");
    })
    .catch((err) => alert(err))
    .finally(() => setPending(false)); // ← manually re-enable UI
}
```

Every state transition is hand-coded. Compare to HTMX's `hx-indicator` + `hx-disabled-elt` — same UX, zero JS.

HTMX: Project Structure

```
ls-htmx/
├── src/
│   ├── db.ts          # Same in-memory store
│   └── index.ts       # Express — routes return HTML fragments
└── views/
    ├── pages/
    │   └── index.pug              # Full page (initial load only)
    └── components/
        ├── wrapperFormList.pug    # Swap target: form + list combined
        ├── inputForm.pug          # Form with hx-* attributes
        └── todoList.pug           # List items with hx-* attributes
```

Same stack as MPA. The key difference: routes return HTML fragments instead of redirecting.

HTMX: One Script Tag

`views/pages/index.pug` — the entire client-side JS dependency:

```
head
  script(
    src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.10/dist/htmx.min.js"
    crossorigin="anonymous"
  )
```

No `npm install` for the frontend. No bundler. No build step. ~14 KB total.

HTMX: Swap Target Architecture

`views/pages/index.pug` — rendered only once, on first load:

```
body(class="container")
  main(class="container")
    a(href="/")
      h1 Todo (HTMX)
    div(id="wrapperFormList")
      include ../components/wrapperFormList.pug
    div(class="spinner-wrapper")
      div(id="spinner" class="lds-dual-ring htmx-indicator")
```

After the first load, only `#wrapperFormList` is ever replaced. The heading and spinner are permanent fixtures — never re-fetched.

HTMX: The Fragment Wrapper

`views/components/wrapperFormList.pug` — exists purely to give HTMX a swap target:

```
div(id="todoform")
  include ../components/inputForm.pug
div(id="todolist")
  include ../components/todoList.pug
```

This extra template is the price of no-refresh updates. The MPA never needs it. The server now must know how to render both a full page (initial load) and this reusable fragment (subsequent updates).

HTMX: Declarative Loading State

`views/components/inputForm.pug`:

```
form(
  hx-post="/create"
  hx-target="#wrapperFormList"
  hx-indicator="#spinner"
  hx-disabled-elt="find input[type='text'], find button"
)
  input(type="text" name="todoText")
  button(type="submit") Submit
```

- `hx-indicator="#spinner"` — shows the spinner while the request is in-flight
- `hx-disabled-elt` — disables inputs and buttons automatically
- **Zero JavaScript written**. The SPA's `setPending(true/false)` logic doesn't exist here.

HTMX: Server Returns HTML Fragments

`src/index.ts` — instead of redirecting, every route renders the fragment:

```
app.post("/create", async (req, res) => {
  await createTodos(req.body?.todoText ?? "");
  const todos = await getTodos();
  res.render("components/wrapperFormList", {
    // ← HTML fragment
    todos,
    mode: "ADD",
    message: "",
    curTodo: blankTodo,
  });
});
```

HTMX: Server Returns HTML Fragments

HTMX receives the HTML and swaps it into `#wrapperFormList`. No JSON. No client-side state management. The server is the single source of truth for both data and UI state.

HTMX: The Cancel Endpoint

Cancel is just fetching the "clean" ADD-mode fragment from the server — no client-side state reset needed:

```
// src/index.ts
app.get("/cancel", async (req, res) => {
  const todos = await getTodos();
  res.render("components/wrapperFormList", {
    todos,
    mode: "ADD",
    message: "",
    curTodo: blankTodo,
  });
});
```

HTMX: The Cancel Endpoint

```
//- inputForm.pug
button(class="contrast" hx-get="/cancel" hx-target="#wrapperFormList") Cancel
```

The server determines what "cancelled" looks like. The client just requests it.

A REST Paradox

	SPA backend	HTMX backend
HTTP verbs	`GET / PUT / PATCH / DELETE`	`POST /edit`, `POST /delete`, `GET /cancel`
Response	JSON — client needs out-of-band API knowledge	HTML — response *is* the next set of possible actions

- SPA satisfies "REST" by convention but violates Fielding's **HATEOAS** constraint
- HTMX looks verb-sloppy but is fully **hypermedia-driven** ✅

> *"Most REST APIs ignore HATEOAS"* — Roy Fielding, 2008